

Security & Observability Design

SALESTORM applies security and observability as cross-cutting concerns across the API Gateway, microservices, databases, message broker, payment integration, and administrative interfaces.

1. Security Design

Transport Security

- TLS 1.2+ is enforced for all external and internal communication.
- mTLS is used for service-to-service communication where required.
- HTTP traffic is redirected to HTTPS.
- Internal services are not directly exposed to the public internet.

Authentication and Authorization

- JWT-based authentication is enforced at the API Gateway.
- Tokens are short-lived to reduce the impact of credential compromise.
- Role-Based Access Control (RBAC) separates permissions for CUSTOMER, OPERATOR, and ADMIN.
- Administrative and simulation APIs require appropriate roles.
- Services validate identity and authorization before performing protected operations.

Edge Protection

- WAF protects public endpoints against common web attacks.
- API Gateway applies rate limiting and concurrency controls.
- Request size limits and schema validation prevent malformed or oversized requests.
- Bot and automated-abuse detection can be applied to flash-sale endpoints.
- The architecture can support a waiting-room or CAPTCHA mechanism during extreme traffic spikes.

Abuse Prevention

- A customer can purchase a maximum of one unit of the flash-sale product.
- Per-user and per-IP throttling limits automated request flooding.
- Idempotency keys prevent repeated requests from creating duplicate business operations.
- Suspicious traffic patterns are monitored through gateway metrics and security logs.

Secrets Management

- Database credentials, JWT secrets, payment credentials, and API keys are stored in a dedicated secrets manager such as Vault or a cloud KMS.
- Secrets are injected at runtime rather than stored in source code or container images.
- Secrets and credentials are never written to application logs.

Payment Security

- Card information is handled through the external payment gateway using tokenization.
- SALESTORM does not store raw PAN or CVV data.
- Payment webhooks are validated using provider signature verification.
- Payment operations use idempotency keys to prevent duplicate charges.
- Payment provider references are stored for reconciliation rather than sensitive card information.

Data Protection

- Sensitive data is encrypted at rest.
- Database access follows least-privilege principles.
- Separate service accounts are used for different databases and operations.
- Administrative actions are recorded in an audit log.
- Access to production data is restricted and monitored.

Secure Logging

Logs must never contain:

- Passwords
- JWTs or access tokens
- Payment tokens
- PAN or CVV
- Secret keys
- Other sensitive authentication credentials

2. Observability Design

SALESTORM uses metrics, structured logs, distributed tracing, and alerts to provide end-to-end visibility across the high-concurrency purchase workflow.

Metrics

Tooling: Prometheus + Grafana

Key metrics include:

- Requests per second
- Request latency: p50, p95, and p99
- HTTP error rate
- Reservation success rate
- OUT_OF_STOCK rate
- Payment success and failure rate
- Payment UNKNOWN count
- Duplicate-request rate
- Idempotency conflict rate
- Inventory database connections and latency
- Transaction failures
- Transactional Outbox backlog
- Message consumer lag
- Dead Letter Queue depth
- Circuit-breaker state
- Order creation delay

The most important business metric is **successful reservations versus available inventory**, because inventory correctness is a primary system invariant.

Structured Logs

Tooling: ELK / OpenSearch

Application logs use structured JSON and include relevant correlation fields such as:

timestamp
service
request_id
trace_id
customer_id
reservation_id
payment_id
order_id
event_id
status
error_code
latency

This allows a single purchase attempt to be traced across the API Gateway, Inventory, Payment, Broker, and Order services. Sensitive credentials and payment data are excluded from logs.

Distributed Tracing

Tooling: OpenTelemetry + Jaeger

Tracing follows the complete purchase path:

Gateway → Checkout → Inventory → Payment → Broker → Order

Trace IDs and correlation IDs are propagated across synchronous API calls and asynchronous events. This helps identify whether latency or failures originate from the gateway, inventory database, payment provider, message broker, or downstream order processing.

Alerting

Tooling: Alertmanager Critical alerts include:

- **Oversell detected → immediate page**
- Inventory conservation mismatch
- High reservation failure rate
- Abnormal payment failure or UNKNOWN rate
- Increasing consumer lag
- Dead Letter Queue depth greater than zero
- Transactional Outbox backlog
- Circuit breaker remaining open
- Database connection exhaustion
- API error-rate SLO breach
- Significant p99 latency increase

Alerts are prioritized according to business impact, with inventory integrity and payment consistency receiving the highest severity.

Audit and Operational Visibility

Administrative actions, configuration changes, and critical recovery operations are recorded in an audit trail. The operations dashboard provides real-time visibility into:

- Flash-sale traffic
- Inventory state
- Reservation outcomes
- Payment outcomes
- Order creation
- Broker health
- Consumer lag
- Failed events
- System latency and error rates